Algorithms and an Exact Cross-Language Toolkit for VLSI Physical Design Automation

Geometry, primal-dual routing, prescribed-skew clocking, and verification

Wai-Shing Luk

physdes

October 11, 2026

Section 1

Outline

Outline

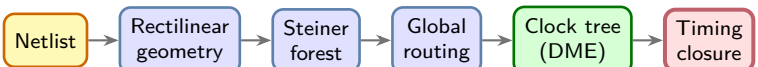
1. Outline
2. Motivation
3. Rectilinear Geometry
4. Primal-Dual Steiner Forest
5. Global and FPGA Routing
6. Clock Tree Synthesis and Timing
7. Engineering and Verification
8. Wrap-up

Section 2

Motivation

Physical Design in One Slide

- A netlist of $\sim 10^9$ objects becomes the polygons a foundry can print.
- Every stage — partitioning, placement, routing, clocking, timing — is an NP-hard combinatorial or geometric problem.
- This talk: an **exact**, dependency-light toolkit in Python, C++20 and Rust.



Three Domain Rules 📐

- **Integer coordinates.** Never floating point: arbitrary precision in Python, explicit fixed width in C++/Rust.
- **Rectilinear geometry.** Axis-parallel edges, plus integer 45-degree abstract lines.
- **Scale.** Billions of objects, but small exceptions: polygons ≤ 100 vertices, nets a few pins, at most 20 metal layers.
- **Keep algorithms simple** — simple $\neq$ easy.

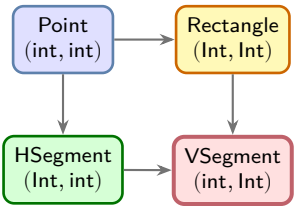
Why Exact Integer Geometry 🔢

- Rounding is not merely imprecise, it is **illegal**: two collinear edges miss, and a design-rule check fails.
- Integers make the predicates exact: overlap, containment, minimum distance, orientation.
- The price is overflow discipline in fixed-width C++ and Rust.

Section 3

Rectilinear Geometry

- A single generic `Point<T1,T2>` yields the whole family of axis-parallel shapes:



- `Interval` and `Point` compose; a 3D point is `Point(Point(x,y), z)`.

Set-Like Operations 🔍

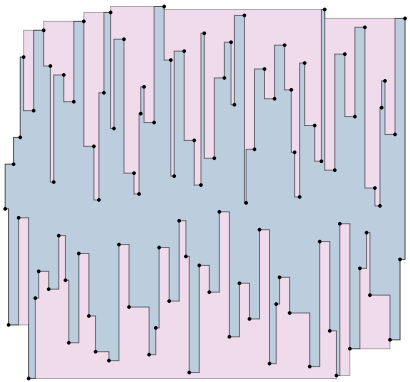
- `overlap(lhs,rhs)` — structural dispatch: use the member if present, else treat as scalar.
- `hull` gives the bounding box; `enlarge` offsets; `blocks` tests "touch without containing".
- One definition in each language: Python `hasattr`, C++20 `requires`, Rust trait bounds.

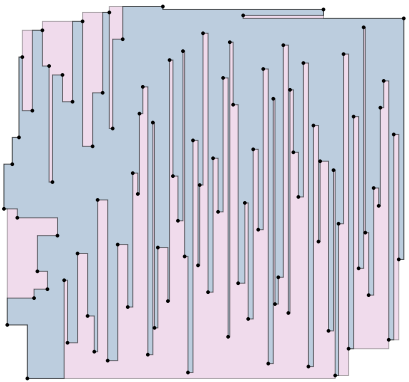
Rectilinear Polygons

- Representation: an origin plus edge vectors — translation is $O(1)$.
- Signed area by the shoelace formula; the sign gives the orientation.
- Point inclusion by ray casting ($O(n)$).

Convex Hull by Two Monotone Passes

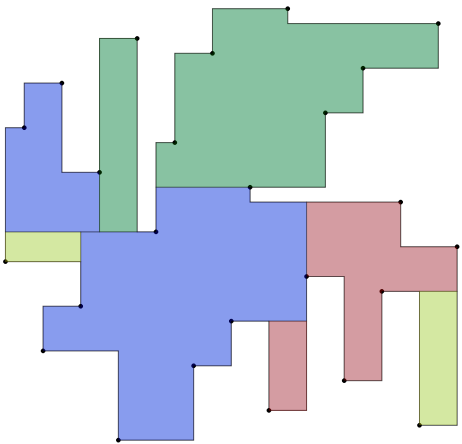
- A rectilinear polygon is convex **iff** it is both x- and y-monotone.
- Two linear passes: reduce to x-monotone, then to y-monotone.





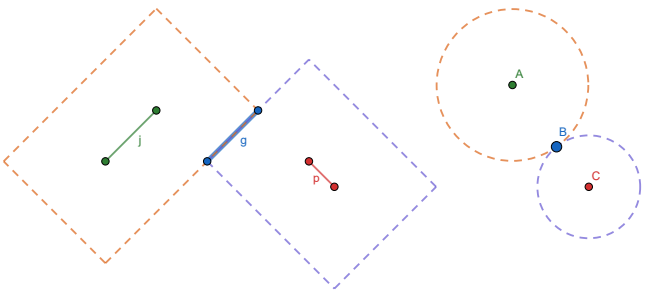
Convex Decomposition ✂

- Recursive cuts at concave vertices into non-overlapping convex pieces.
- $O(n^2)$ worst case, $O(n \log n)$ when the cuts are balanced.
- The cut primitive is what a router uses to negotiate an obstacle.



Manhattan Arcs ➡

- In $u = x - y,\ v = x + y$ the L_1 unit ball becomes an axis-aligned square.
- Intersecting two **enlarged** Manhattan arcs yields the parent's region — the L_1 analogue of intersecting two circles [1].



Section 4

Primal-Dual Steiner Forest

The Steiner Forest Problem

$$\min_{F} \sum_{e \in E} w_e \, x_e \quad \text{s.t.} \quad \sum_{e \in \delta(S)} x_e \geq 1, \quad \forall S \in \mathcal{S}_i, \quad x_e \in \{0,1\}$$

- Connect k terminal pairs at minimum cost; **NP-hard** and APX-hard.
- The undirected cut relaxation has a *moat* dual: no edge crossed by moats wider than its cost.

Primal-Dual Construction 🛠️

- Grow all **active components** uniformly until some edge becomes tight; add it and merge; repeat [2].
- The uniform step is $\Delta_e = \dfrac{c_e - \mathrm{paid}(e)}{\mathrm{num_active}(e)}$.
- Union-find keeps each component query at $O(\alpha(n))$; reverse-delete prunes cycles.

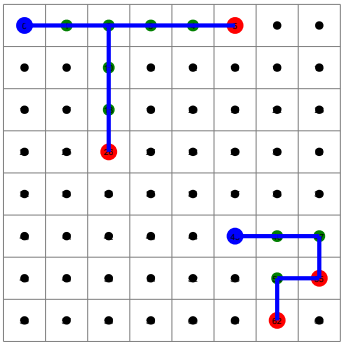
Why the Ratio Is Two ⚖️

$$c(F') = \sum_S y_S \, |F' \cap \delta(S)| \;\le\; 2 \sum_S y_S \;\le\; 2 \; \mathrm{OPT}$$

- Reverse-delete makes F' a forest, so the degrees of the active components are bounded.
- Worst observed ratio on SteinForestLib: **1.269**.

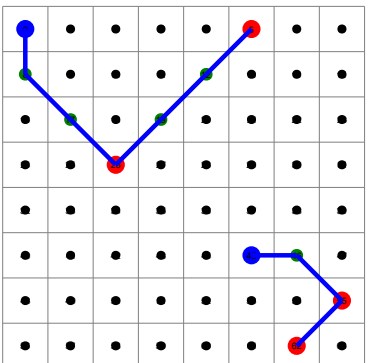
Experimental Quality

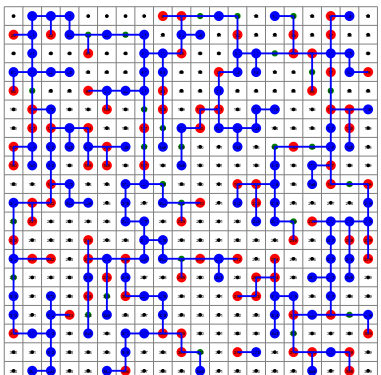
- PD-SF: ratio 2, worst observed 1.269, fastest.
- Greedy: $\Omega(\log n)$; randomized: $O(\log n)$.



A Forest at Work

- **Left:** the primal-dual forest extends to 45-degree edges.
- **Right:** at 20×20 with 150 pairs the forest becomes a density pattern — a reminder that per-net structure does not scale.





Section 5

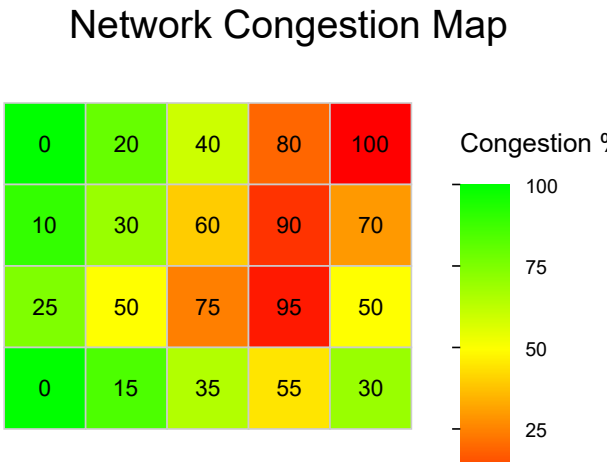
Global and FPGA Routing

Global Router: Three Strategies

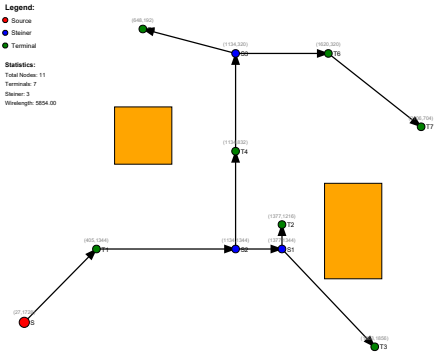
- `route_simple` — nearest existing node, $O(nm)$.
- `route_with_steiners` — share segments, $O(nmk)$.
- `route_with_constraints` — wirelength budget $\alpha\cdot$ worst, $\alpha \in [0.5, 2]$.
- Output: a Steiner-point-guided tree plus a congestion map for the placer.

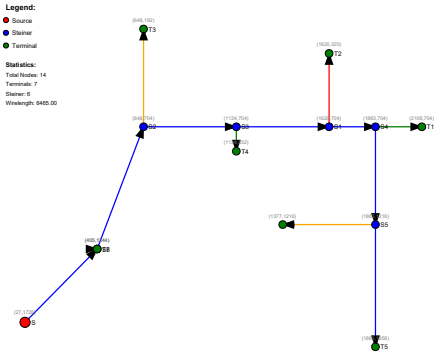
Congestion: A Renderer, Not a Model

- The map is a caller-supplied utilization grid drawn as a green-to-red heat map.
- It is a **visualization**; there is no capacity or overflow model behind it.



Keepouts and 3D





- Rectangular **keepouts** are avoided via `contains`/`blocks`.
- 3D is just a nested point `Point(Point(x,y),z)`; layers cost vias.

- Routing on a fixed fabric: maze, A*, **Pathfinder** (negotiated rip-up), VPR, ROAD (bump-and-refit), SAT.
- ROAD's pruning sped the base search $604\times$; SAT can *prove* unroutability.

router	paradigm	minimum tracks
VPR routability	negotiated rip-up	minimal
VPR timing	+ Elmore	$\sim 5\%$ more
ROAD	bump-and-refit	minimal
SAT	CNF satisfiability	$\sim 25\%$ more

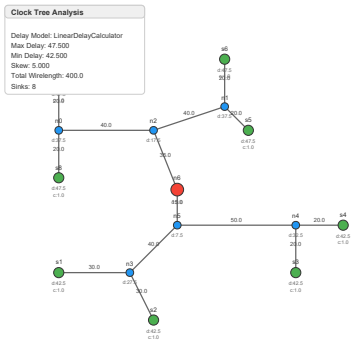
Section 6

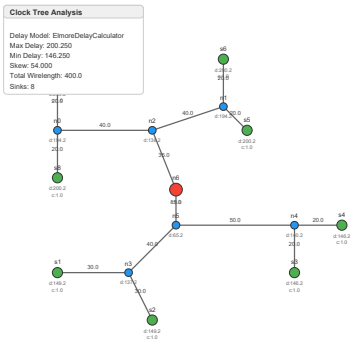
Clock Tree Synthesis and Timing

Deferred-Merge Embedding ⏰

- Bottom-up: annotate every node with its **merging segment**.
- Top-down: place each node at the nearest point of its segment [3].
- Handles **prescribed skew** (a generalization of zero-skew); $O(n \log n)$, linear-time median selection in all three implementations.

Two Delay Models



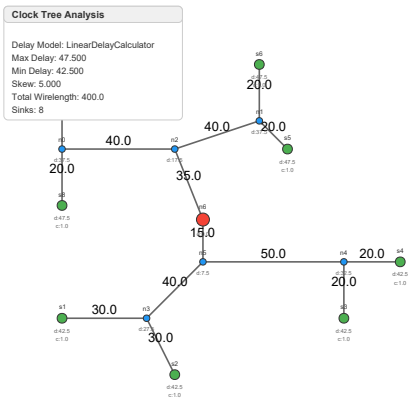


- Linear: $D = kL$; Elmore: $D = R\,(C/2 + C_{\text{load}})$.
- The model is a swappable `DelayCalculator` strategy; the tree is unchanged.

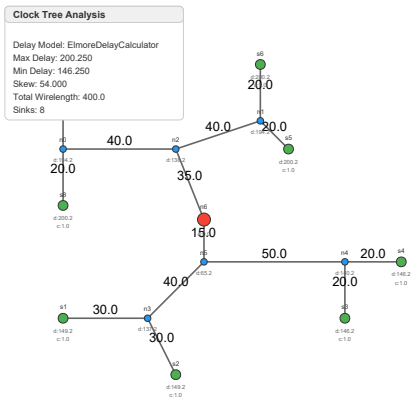
The Model Changes the Numbers

- Same topology, two delay models: the absolute delay (and residual skew) differ.
- That is why the model is a *strategy*, not a constant.

Linear Delay Model



Elmore Delay Model



$$T_{\text{clk}} \geq T_{ckq} + T_{\text{logic}} + T_{\text{setup}} - T_{\text{skew}}, \qquad T_{ckq} + T_{\text{logic}} \geq T_{\text{hold}} + T_{\text{skew}}$$

- Slack = required − arrival; close the worst and total negative slack.
- **Useful skew** is a linear program; **delay padding** fixes residual hold violations.

Section 7

Engineering and Verification

An Exact Cross-Language Toolkit 🌍

- Python, C++20 and Rust produce **bit-identical** results.
- Cross-language harnesses pin shared oracle constants in each test suite.
- It caught silent bugs a single port hides: a mutating elongation routine, a set-order tie-break.

Removing Asymptotic Waste

- Optimize **asymptotics**, prove the output unchanged with a *differential oracle*.

component	change	speedup
TSP 2-opt	$O(n^3) \to O(n^2)$ per sweep	$99\times$
cycle cover	$O(kE) \to O(E)$	$\sim 10^4\times$
tree string	$O(n^2) \to O(n)$	$8.6\times$

Scope and Threats to Validity 🎯

1. **Synthetic only** — no foundry data; the claims are algorithmic, not signoff.
2. **Scalability** — Steiner forest $O(V^2)$, decomposition $O(n^2)$; $n > 10^3$ is open.
3. **Model fidelity** — two first-order delay models; no SSTA, crosstalk or lithography [4].
4. **Flow coverage** — *global* routing only; no detailed routing or DRC [5].
5. **Parity cost** — three test suites, but optimization stays output-preserving.

Section 8

Wrap-up

Conclusion

- Exact integer geometry, primal-dual algorithms, and verifiable cross-language ports.
- Simple, reproducible, and loud about its own failure modes.
- Open problems: measured-silicon validation, sub-quadratic full-chip scaling, advanced delay and lithography models.

Q&A

- Code: `physdes` and `netlistx` under the `luk036` organization.
- Thank you!

[1] D.-T. Lee and C.-K. Wong, "Voronoi diagrams in L_1 (L_∞) metrics with two-dimensional storage applications," *SIAM J. Comput.*, vol. 9, no. 1, pp. 200–211, 1980.

[2] A. Agrawal, P. Klein, and R. Ravi, "When trees collide: An approximation algorithm for the generalized Steiner problem on networks," *SIAM J. Comput.*, vol. 24, no. 3, pp. 440–456, 1995.

[3] T.-H. Chao, Y.-C. Hsu, J.-M. Ho, K. D. Boese, and A. B. Kahng, "Zero skew clock routing with minimum wirelength," *IEEE Trans. Circuits Syst. II: Analog Digit. Signal Process.*, vol. 39, no. 11, pp. 799–814, 1992.

[4] C. Visweswariah, K. Ravindran, K. Kalafala, S. G. Walker, and S. Narayan, "First-order incremental block-based statistical timing analysis," in *Proc. 41st annual design automation conf. (DAC)*, 2004, pp. 331–336. doi: 10.1145/996566.996663.